

Laboratório 404: um jogo 3D educacional com uma IA local restrita ao mundo — arquitetura, processo incremental e validação automatizada

William da Silva Vianna
Instituto Federal Fluminense

Setembro de 2026

Resumo

Jogos educacionais podem se beneficiar de grandes modelos de linguagem (LLMs) capazes de dialogar com o jogador, mas integrar esses modelos a um jogo real impõe desafios de segurança (a IA não deve controlar o jogo), de desempenho (inferência local em hardware modesto) e de engenharia (manter o processo rastreável em um desenvolvimento de autor único assistido por agentes de IA). Este artigo relata o projeto, a implementação e a validação do **Laboratório 404**, um jogo 3D educacional/ficcional no qual a IA **ARIA** administra uma instalação subterrânea e conversa com o jogador sem nunca controlar o estado do mundo. O protótipo foi construído em seis provas de conceito incrementais (POC 1 a POC 6) sobre Godot 4.5, Blender 5.2.1, um adaptador Python (FastAPI) e o Ollama com modelo local; as respostas da IA são restritas a cinco intenções validadas em duas camadas, com *fallback* textual. A validação combinou especificação rastreável, suíte automatizada *headless* e evidências visuais. Resultados: 211 verificações Godot e 16 testes Python aprovados; cena 3D com 342 nós e 37 materiais (1.018,7 KB em GLB) executando a 60 FPS (limitado por *vsync*) em GPU integrada AMD Radeon Graphics a 1280×720; respostas de ARIA com o modelo local `llama3.1:8b` entre 12,9 s e 22 s em três medições. As limitações incluem ausência de estudo com usuários, validação manual de hardware pendente e não execução do modelo-alvo `llama3.2:3b`. Os artefatos (código, especificação, capturas e vídeo de demonstração) são públicos.

Palavras-chave: jogos sérios; jogos educacionais; grandes modelos de linguagem; inferência local; Godot; desenvolvimento orientado a especificação.

1 Introdução

Jogos sérios são jogos cujo propósito principal não é o puro entretenimento (Abt, 1970) e que, nas últimas duas décadas, consolidaram-se como instrumentos de educação, treinamento e simulação (Zyda, 2005; Laamarti et al., 2014). A literatura sobre aprendizagem sustenta que bons jogos incorporam princípios pedagógicos — identidade, experimentação ativa, problemas bem ordenados e retroalimentação imediata — que podem ser aproveitados também na educação formal (Gee, 2003). Construir um jogo com esse propósito, porém, exige mais do que uma boa ideia: exige arquitetura, processo e evidências de que o artefato realmente funciona.

Nos últimos anos, grandes modelos de linguagem (LLMs) passaram a ocupar papel de destaque em jogos, seja gerando conteúdo, seja animando personagens ou apoiando o próprio desenvolvimento (Gallotta et al., 2024). Os resultados são promissores, mas trazem dois problemas práticos. O primeiro é de *controle*: agentes que decidem e executam ações podem alterar o estado do jogo de formas imprevisíveis — em *Generative Agents*, os personagens planejam, refletem e agem com grande autonomia (Park et al., 2023) e, em *Voyager*, o agente escreve e executa código para progredir (Wang et al., 2023); ambos dependem de modelos de grande porte remotos ou de infraestrutura substancial. O segundo é de *reprodutibilidade*: é raro que um projeto de jogo com

IA descreva, com o mesmo cuidado, o processo de desenvolvimento, os critérios de aceitação e as evidências de teste.

Este artigo enfrenta esses dois problemas em um caso real: o **Laboratório 404**, um jogo 3D educacional/ficcional no qual uma IA local, **ARIA**, administra uma instalação subterrânea e conversa com o jogador — mas nunca controla o jogo. O protótipo foi desenvolvido em seis provas de conceito incrementais (POC 1 a POC 6) sobre Godot 4.5 (gameplay), Blender 5.2.1 (assets 3D), um adaptador Python/FastAPI e o Ollama com modelo local. A IA recebe contexto do jogo e devolve apenas texto e uma intenção restrita a cinco valores validados em duas camadas independentes; qualquer desvio vira **NONE** e o jogo usa uma resposta de *fallback*. O desenvolvimento foi documentado como especificação executável (requisitos funcionais e não funcionais, critérios de aceitação no formato DADO/QUANDO/ENTÃO e matriz de rastreabilidade) e validado por uma suíte automatizada *headless*.

O trabalho parte de três perguntas de pesquisa:

- **RQ1 (arquitetura):** como integrar uma LLM local a um jogo 3D de modo que ela enriqueça o mundo e a narrativa sem, em nenhum momento, controlar o estado do jogo?
- **RQ2 (viabilidade):** é possível manter jogabilidade a 60 FPS e conversa local com latência tolerável em um computador sem GPU dedicada?
- **RQ3 (processo):** como conduzir e documentar esse desenvolvimento — de autor único, assistido por agentes de IA — de forma rastreável e reproduzível?

As contribuições são: (i) um padrão de arquitetura de *IA diegética com capacidade limitada*, com dupla validação de intenções e *fallback* (Seção 3); (ii) um processo incremental orientado a especificação, com evidências automatizadas e visuais; (iii) uma implementação completa e aberta — código, especificação, capturas e vídeo de demonstração — disponível publicamente (Vianna, 2026a,b); e (iv) resultados empíricos de execução em GPU integrada, incluindo as latências medidas da conversa com o modelo local.

O restante do artigo está organizado assim: a Seção 2 revisa os trabalhos relacionados; a Seção 3 descreve o processo, a arquitetura, o pipeline de assets, a integração da IA e a estratégia de validação; a Seção 4 apresenta os resultados; a Seção 5 discute interpretação, comparação, limitações e implicações; e a Seção 6 conclui.

2 Fundamentação teórica e trabalhos relacionados

2.1 Jogos sérios e educacionais

O termo *jogos sérios* popularizou-se a partir da definição de Abt: jogos com um propósito educacional explícito, que não são feitos apenas para divertir — embora possam ser divertidos (Abt, 1970). A pesquisa em aprendizagem com jogos mostrou que bons jogos já incorporam, de forma orgânica, princípios que a escola persegue: identidade, experimentação, risco controlado, regras claras e retroalimentação constante (Gee, 2003). A visão de simulação virtual como base para jogos, e dos jogos como ferramenta de treinamento, foi consolidada na transição de simulação para realidade virtual (Zyda, 2005). Revisões mais recentes organizam o campo por aplicação (saúde, defesa, educação, planejamento) e apontam que a avaliação de impacto segue sendo um desafio metodológico (Laamarti et al., 2014). [REFERÊNCIA NECESSÁRIA: diretrizes contemporâneas de avaliação de jogos sérios em contextos escolares.]

2.2 Grandes modelos de linguagem em jogos

O levantamento de Gallotta *et al.* mapeia os papéis que LLMs podem assumir em jogos — geração de conteúdo, personagens, apoio ao projeto, testes — e discute limites práticos, como

custo, latência e confiabilidade (Gallotta et al., 2024). Duas limitações recorrentes interessam diretamente a este trabalho: (a) modelos grandes exigem infraestrutura relevante ou serviços externos; e (b) quanto maior a autonomia concedida ao modelo, maior a superfície de risco sobre o estado do jogo.

2.3 Agentes generativos e autonomia

Generative Agents propôs agentes que armazenam experiências em linguagem natural, sintetizam reflexões e planejam comportamento, produzindo comportamento social emergente e verossímil em um ambiente interativo (Park et al., 2023). *Voyager* levou a autonomia adiante em Minecraft: um currículo automático, uma biblioteca de habilidades em código e um mecanismo iterativo de prompting com retroalimentação do ambiente; o agente aprende continuamente sem ajuste de parâmetros, consultando um modelo externo (Wang et al., 2023). Esses trabalhos demonstram o potencial da autonomia, mas também a distância entre esse desenho e um jogo educacional em que cada ação precisa ser previsível, auditável e barata de executar. Neste artigo, a escolha é deliberadamente oposta: a IA conversa, comenta e sugere; quem age é sempre o motor do jogo.

2.4 Inferência local

Executar o modelo na própria máquina elimina dependência de serviços externos, dispensa chaves de API e mantém o conteúdo no computador do usuário; o Ollama é uma ferramenta consolidada para esse cenário (Ollama, 2026). O custo é a latência: modelos menores rodam em hardware modesto, mas geram respostas em escala de segundos, o que restringe seu uso a interações conversacionais (por turnos) em vez de reações em tempo real. [REFERÊNCIA NECESSÁRIA: estudo comparativo de latência de LLMs locais em hardware sem GPU dedicada.]

2.5 Processo de desenvolvimento

Práticas como desenvolvimento orientado a testes e a especificações propõem que critérios verificáveis sejam escritos antes da implementação. [REFERÊNCIA NECESSÁRIA: fontes canônicas sobre desenvolvimento orientado a especificação.] No contexto deste projeto, essas práticas foram adaptadas a um fluxo de trabalho assistido por agentes de IA: a especificação, os requisitos e os critérios de aceitação são os artefatos que restringem cada iteração de implementação, e a suíte automatizada é o juiz comum entre autor humano e agente. O uso de servidores MCP (*Model Context Protocol*) para dar ao agente acesso controlado a ferramentas — no caso, a cena do Blender — é parte desse fluxo (Vianna, 2026a).

2.6 Síntese e lacuna

A literatura concentra-se, de um lado, em agentes autônomos e capazes de agir (Park et al., 2023; Wang et al., 2023) e, de outro, em visões amplas sobre o papel das LLMs em jogos (Gallotta et al., 2024). O que este trabalho investiga é a combinação menos explorada: *uma IA diegética, com capacidade formalmente limitada, executada localmente e integrada a um jogo educacional completo, com processo de desenvolvimento documentado e evidências de validação*. É nessa lacuna que se posicionam as contribuições descritas na Seção 3.

3 Materiais e métodos

3.1 Estratégia de desenvolvimento

O projeto seguiu um processo orientado a especificação, adaptado ao contexto de autor único assistido por agentes de IA. Antes do código, foram fixados: uma constituição com princípios não negociáveis; um roadmap de seis provas de conceito; uma especificação com requisitos funcionais

(FR) e não funcionais (NFR), critérios de aceitação no formato DADO/QUANDO/ENTÃO e uma matriz de rastreabilidade requisito–teste; e regras permanentes de interação para agentes de IA (Vianna, 2026a).

Duas regras estruturaram o trabalho. A primeira é a *Regra de Ouro*: cada POC termina em uma versão executável e demonstrável, com caminho de teste reproduzível — o protótipo nunca fica quebrado entre etapas. A segunda é a proibição de índices de botão no gameplay (FR-001/NFR-005): o jogo consome apenas ações semânticas de um *Input Map* (`move_forward`, `interact`, `pause` e outras), o que manteve o mesmo código funcional para teclado e para o adaptador USB de PlayStation 1, cuja enumeração de eixos e botões varia conforme o hardware.

Os agentes de IA trabalharam dentro dessas restrições: a especificação e os critérios de aceitação funcionavam como contrato de cada iteração e a suíte automatizada, como juiz comum. Um servidor MCP (*Model Context Protocol*) no editor dava ao agente acesso controlado à cena do Blender para modelagem assistida (Vianna, 2026a).

3.2 Ambiente e ferramentas

A Tabela 1 resume o ambiente de desenvolvimento e execução, verificado por comando na máquina de referência. As versões reportadas são exatamente as que produziram os resultados da Seção 4; o projeto mantém um passo de importação de recursos (GLB e áudio) executado por linha de comando, necessário na primeira execução de cada clone.

Tabela 1: Ambiente de desenvolvimento e execução (verificado em 2026-09-10).

Item	Versão / configuração
Sistema operacional	Ubuntu 24.04.4 LTS
CPU / GPU	AMD Ryzen 7 5700U com Radeon Graphics (GPU integrada)
Memória	30 GiB
Motor de jogo	Godot 4.5 (4.5.stable.official.876b29033), renderer <code>gl_compatibility</code>
Modelagem 3D	Blender 5.2.1 LTS
Linguagem do adapter	Python 3.12.3
Bibliotecas do adapter	FastAPI 0.141.1; Uvicorn 0.52.4; Pydantic 2.13.5; requests 2.34.2
Inferência local	Ollama; modelo-alvo <code>llama3.2:3b</code> ; validação com <code>llama3.1:8b</code>
Formato de assets	glTF 2.0 binário (GLB)

O motor de jogo (Godot Engine, 2026) concentra gameplay, física, interface, áudio e persistência; o Blender (Blender Foundation, 2026) produz os assets exportados em GLB (Khronos Group, 2026); o adaptador usa FastAPI (FastAPI, 2026), Pydantic (Pydantic, 2026) e Uvicorn (Encode, 2026); e a inferência local é feita pelo Ollama (Ollama, 2026). Os efeitos sonoros são gerados por um script do próprio projeto, sem conteúdo de terceiros.

3.3 Arquitetura do sistema

A Figura 1 apresenta a arquitetura. O jogo é um processo Godot que consome assets exportados do Blender e consome *input* de teclado e joystick por ações semânticas. A conversa com a IA nunca é feita diretamente pelo motor: passa por um processo separado (o adaptador), acessível por HTTP em `localhost`, que valida a entrada, monta o prompt com a personalidade versionada e o estado do jogo, chama o Ollama e valida a saída. Separar a IA do jogo em processos distintos tem duas consequências práticas: o jogo continua jogável com a IA desligada (respostas de *fallback*) e a falha de qualquer elo não derruba a aplicação.

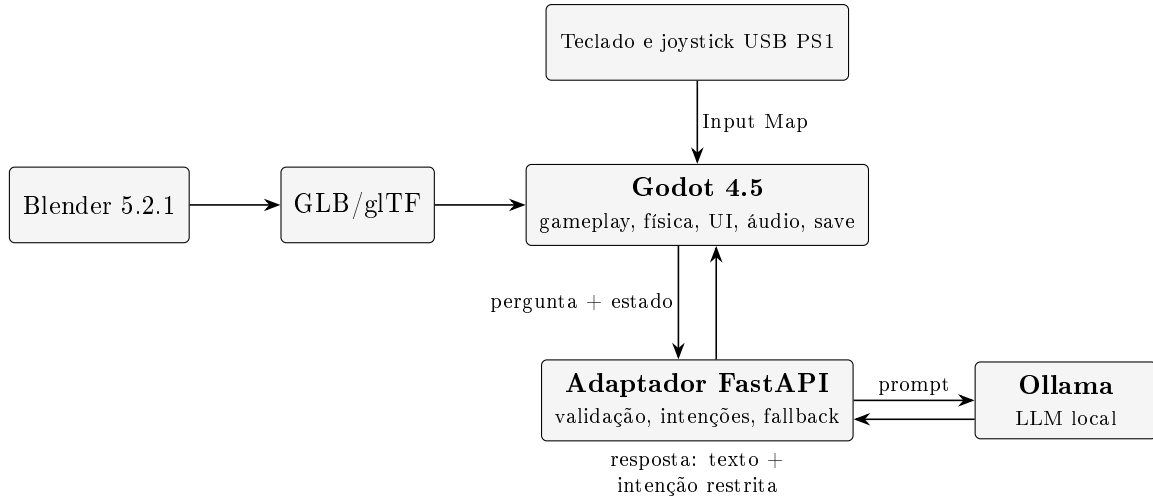


Figura 1: Arquitetura do Laboratório 404. A IA vive fora do processo do jogo: o adaptador é a fronteira onde entrada e saída são validadas.

3.4 Pipeline de assets (Blender para GLB)

A cena 3D foi modelada no Blender sob convenções fixas: 1 unidade do Blender equivale a 1 metro (NFR-002); objetos usam prefixos por função (**ENV_** para ambiente, **PROP_** para equipamentos, **DEC_** para decoração, **NPC_** para personagens); e a colisão é embutida no próprio asset por meio do sufixo `-col`, que o importador do Godot converte em corpo estático, mantendo a malha visível original para renderização. As transformações são aplicadas antes da exportação, e o GLB resultante é importado pela linha de comando do motor, sem edição manual de cena. O detalhamento progressivo da sala — luminárias, letreiros, sinalização, mobiliário e um NPC completo — foi conduzido nessas convenções e medido ao final (Seção 4).

3.5 ARIA: IA local com intenções restritas

O núcleo do desenho de segurança está na Figura 2. A IA devolve sempre um objeto estruturado com texto, intenção e confiança; a intenção é sanitizada no adaptador e novamente no cliente do jogo. O conjunto permitido é deliberadamente pequeno:

```
ALLOWED_INTENTS = {"HINT", "DIAGNOSE", "LORE",
                   "STATUS", "NONE"}
```

Qualquer valor fora desse conjunto — inclusive ruído devolvido pelo modelo — é convertido para **NONE** (FR-017, CA-010). A resposta também é limitada em forma e conteúdo: mensagem de entrada entre 1 e 2000 caracteres (erro 422 quando inválida), *timeout* de 30 s na chamada ao Ollama e resposta de *fallback* textual em caso de exceção (FR-018, NFR-004). O cliente no jogo espera um pouco mais que o adaptador e, se nada chegar, também usa texto pré-definido — ou seja, a IA pode estar lenta ou ausente sem comprometer a jogabilidade (CA-009). A personalidade de ARIA (identidade, regras, tons de voz e texto de *fallback*) fica em um arquivo JSON versionado, não espalhada em código (FR-020), e o estado do jogo é injetado no prompt como contexto factual. Em nenhum ponto a IA recebe permissão de escrever estado, abrir portas ou alterar física (FR-021, NFR-006).

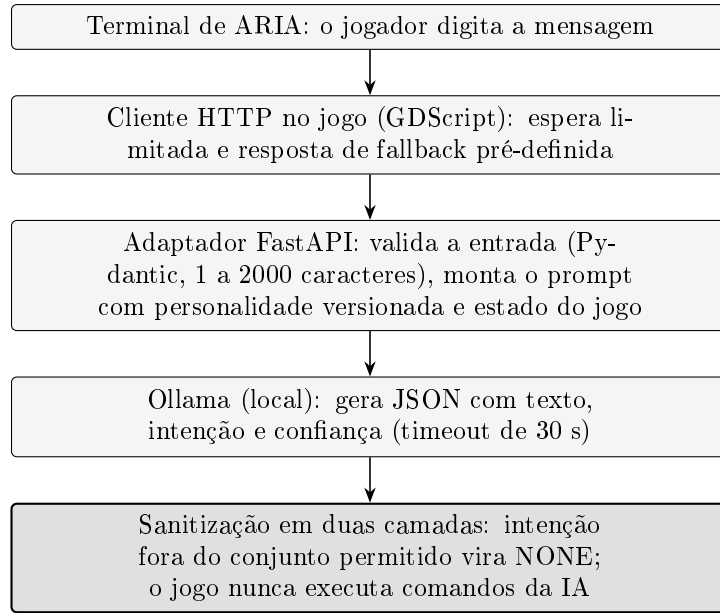


Figura 2: Cadeia de uma mensagem até a resposta. Cada elo falha de forma segura: o pior caso é um texto de fallback, nunca um travamento ou uma ação não prevista.

3.6 Conteúdo e jogabilidade

O arco do vertical slice começa com a chegada do técnico à instalação e um tutorial de controles dentro do jogo; segue pela missão **RESTAURAR_COMUNICACAO** (Figura 3); passa por um corredor que dá acesso a uma ala procedural gerada a partir de uma semente determinística (mesma semente, mesmo laboratório; conectividade garantida por construção) e culmina no núcleo de ARIA, com um epílogo de três escolhas. Ao longo do percurso, o mundo reage ao jogador: luzes mudam conforme o estado da energia, uma câmera de segurança passa a acompanhar o jogador depois que a energia volta, um NPC comenta o estado dos equipamentos e a ARIA muda de tom quando um alerta é ignorado. O progresso é serializável em arquivo local (save/load) e a ambiência sonora é contínua, com efeitos para interações, porta, alarme e erro.

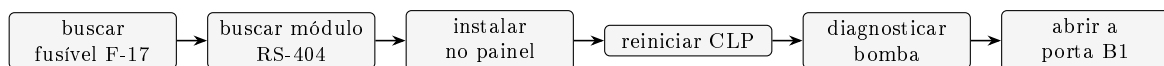


Figura 3: A missão principal em seis passos dependentes e ordenados. Cada passo só é aceito quando o anterior está concluído.

3.7 Validação automatizada e evidências

A validação combina três instrumentos. O primeiro é a suíte de testes *headless*: cada POC tem uma cena de teste que instancia o jogo, executa interações reais (mover, coletar, instalar, conversar, salvar) e verifica o estado resultante, sem depender de janela ou de intervenção manual. O segundo é um conjunto de testes de API em Python que exercita o adaptador com requisições válidas e inválidas, inclusive limites e *fallback*. O terceiro é um arnês de captura de tela: uma cena que posiciona o jogador em poses pré-determinadas, opcionalmente conclui a missão e salva imagens automáticas em 1280×720 — usado para produzir as evidências visuais deste artigo.

Itens que exigem presença física ou julgamento humano são explicitamente separados: operação de mouse com captura de cursor, calibração do adaptador PS1 em hardware real e o playthrough humano de ponta a ponta (CA-013). Esses itens estão declarados como pendentes nos artefatos do projeto, e não são tratados como concluídos por inferência.

4 Resultados

4.1 Artefatos entregues

Ao final do ciclo, as seis provas de conceito estavam implementadas e demonstráveis de forma independente, conforme a Regra de Ouro: fundação jogável; laboratório funcional com missão completa; ARIA integrada por adapter; mundo reativo; laboratório procedural; e o vertical slice com tutorial, persistência, áudio e epílogo. O material público inclui o código-fonte, a especificação com requisitos e critérios de aceitação, as capturas de tela usadas neste artigo e um vídeo de demonstração de 1 min 08 s (1280×768 a 30 fps) que percorre a missão do POC 2 (Vianna, 2026a,b). O conjunto de scripts do jogo soma cerca de 2.400 linhas de GDScript (contagem bruta de linhas, incluindo comentários) e a suíte de testes do motor, cerca de 1.300 linhas.

4.2 Suíte automatizada

A Tabela 2 detalha as verificações por POC. No total, 210 a 211 verificações do motor mais 16 testes do adapter, todas aprovadas na execução de referência. A variação de uma verificação no POC 3 é esperada: com o adapter no ar, um dos casos de *fallback* dá lugar à validação da resposta real do modelo.

Tabela 2: Verificações automatizadas por prova de conceito (suíte *headless*, execução de referência de 2026-09-10).

POC	Escopo validado	Verificações
1	Fundação jogável: input semântico, movimento, câmera, pausa, HUD, diagnóstico de input	77
2	Laboratório funcional: interação, inventário, porta, missão, sala modelada	34
3	ARIA: adapter, terminal, timeout, validação de intenção, fallback	25–26
4	Mundo inteligente: NPC, luzes por energia, tom da ARIA, memória em camadas	21
5	Laboratório procedural: determinismo por semente, conectividade entre módulos	23
6	Vertical slice: tutorial, missão completa, save/load, áudio, epílogo	30
Adapter	API de ARIA (Python): contratos, validação, erros e limites	16
Total		210–211 + 16

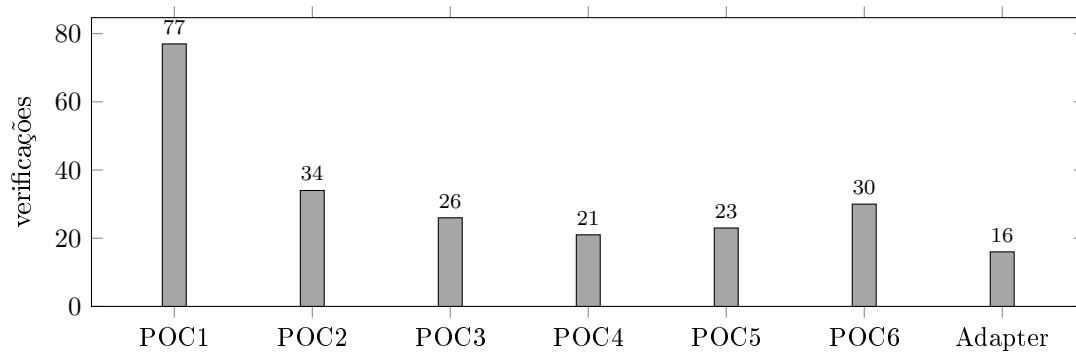


Figura 4: Distribuição das verificações automatizadas por etapa do projeto.

4.3 Cena 3D e desempenho

A sala do laboratório foi exportada em um único arquivo GLB com 342 nós, 37 materiais e 35 objetos de colisão embutidos, totalizando 1.018,7 KB; o emissivo dos materiais usa a extensão `KHR_materials_emissive_strength`, o que permitiu acender telas e sinalizações sem estourar o branco no renderizador compatível. A Figura 5 mostra o resultado em jogo. O desempenho medido foi de 60 FPS limitados por *vsync* a 1280×720 na GPU integrada, inclusive após o detalhamento da cena (342 objetos e 34 elementos de iluminação/letreiros), mantendo o critério NFR-001. Métricas de tempo de quadro por percentil não foram coletadas [MEDICÃO DE FRAME TIME NÃO REALIZADA].

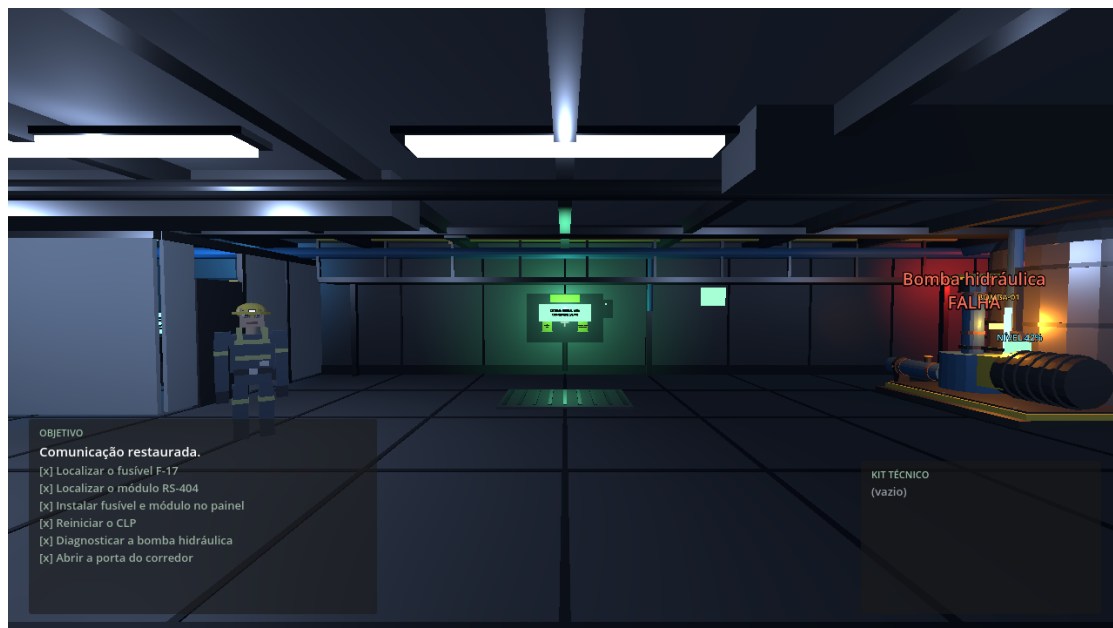


Figura 5: Vista da sala do Laboratório 404 com a energia restaurada: luminárias do teto acesas, câmera de segurança ativa, porta B1 liberada e letreiros das estações. Captura do jogo em 1280×720 (fonte: material do projeto).

4.4 Conversa com ARIA

O critério de aceitação central — receber uma resposta útil ou um *fallback* em tempo limitado, sem travar o jogo — foi validado em três contextos com o modelo local `llama3.1:8b` (Tabela 3). As latências ficaram entre 12,9 s e 22,0 s; a partida fria é o pior caso, e o teste no jogo (Figura 6) passou 5 de 5 verificações de integração. O modelo-alvo declarado na especificação, `llama3.2:3b`,

não estava instalado no ambiente de referência, de modo que os números representam o desempenho com um modelo maior e mais lento — ou seja, são um limite superior conservador para o alvo. Com o adapter ou o modelo fora do ar, o jogo responde com texto de *fallback* e intenção *NONE*, conforme projetado (CA-009).

Tabela 3: Latências medidas da conversa com ARIA usando modelo local `llama3.1:8b`.

Contexto	Intenção	Latência	Observação
Chamada direta ao adapter (partida fria)	DIAGNOSE	22,0 s	confiança 0,9
Terminal de ARIA no jogo	STATUS	16,1 s	captura em docs/images
Cliente no jogo (teste ao vivo)	HINT	12,9 s	5 de 5 verificações

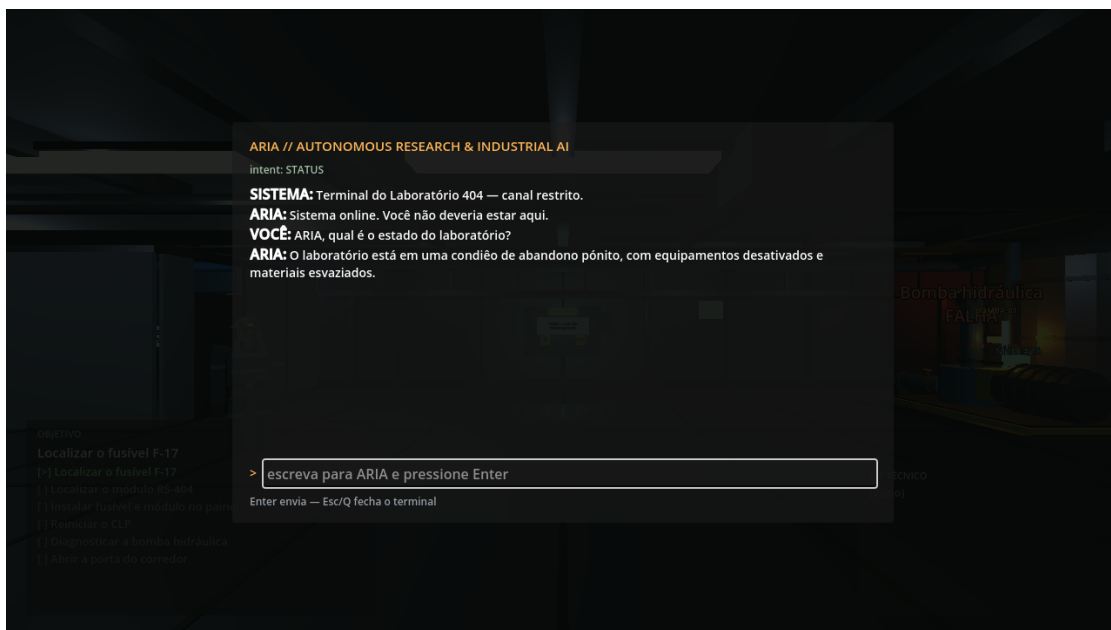


Figura 6: Terminal de ARIA no jogo: a pergunta do jogador e a resposta do modelo local (latência de 16,1 s, intenção *STATUS*). Fonte: material do projeto.

5 Discussão

5.1 Interpretação dos resultados

Os resultados respondem afirmativamente às três perguntas de pesquisa, dentro do escopo de um protótipo. Sobre a arquitetura (RQ1): a combinação de processo separado, contrato estruturado e sanitização em duas camadas mostrou-se suficiente para que a IA participasse do mundo — dando dicas, diagnosticando equipamentos, comentando o estado do laboratório e mudando de tom — sem receber qualquer capacidade de alterar o estado do jogo. O pior caso observado, em todas as falhas induzidas nos testes, foi uma resposta de *fallback*: nunca um travamento, nunca uma ação não prevista.

Sobre a viabilidade (RQ2): a jogabilidade manteve-se em 60 FPS limitados por *vsync* na GPU integrada, com a cena detalhada (342 objetos, 37 materiais), e a conversa local apresentou latência de 12,9 s a 22,0 s com um modelo de 8 bilhões de parâmetros. Essa ordem de grandeza é adequada a uma interação conversacional por turnos — o terminal de ARIA — e inadequada

a reações em tempo real, o que reforça a decisão de projeto de não colocar a IA no laço de controle. Sobre o processo (RQ3): a especificação com critérios verificáveis e a suíte automatizada permitiram que cada etapa fosse aceita ou rejeitada com base em evidência, e não em impressão; as seis etapas fecharam com a suíte verde.

5.2 Comparação com a literatura

O contraste com os trabalhos de referência é deliberado. *Generative Agents* e *Voyager* investigam o que acontece quando se concede ao modelo autonomia de decisão e de execução (Park et al., 2023; Wang et al., 2023); o Laboratório 404 investiga o oposto — quanto valor narrativo e pedagógico se obtém mantendo a IA estritamente consultiva. Essa escolha dialoga com o levantamento de Gallotta *et al.*, que destaca custo, latência e confiabilidade como limites práticos das LLMs em jogos (Gallotta et al., 2024): aqui esses limites não foram eliminados, foram convertidos em restrições de projeto (intenções finitas, *fallback* obrigatório, inferência local). Não há, neste trabalho, comparação quantitativa com os sistemas citados, pois os alvos são diferentes (agência aberta versus assistência limitada) [ESTUDO COMPARATIVO NÃO REALIZADO].

5.3 Limitações e ameaças à validade

- **Estudo de caso único, sem usuários:** o trabalho é uma descrição de projeto e uma validação técnica; não mede aprendizagem nem experiência do jogador. O playthrough humano de ponta a ponta (CA-013) permanece pendente [ESTUDO DE USUÁRIO NÃO REALIZADO].
- **Amostra pequena de latência:** as três medições da Tabela 3 são informativas, não experimentais; não há repetição controlada, variação de carga ou comparação entre modelos. O modelo-alvo `llama3.2:3b` não foi executado.
- **Ambiente:** os números de desempenho valem para a máquina da Tabela 1; a validação em joystick PS1 físico e o uso de mouse capturado seguem manuais. Durante o desenvolvimento, dois atritos de ambiente exigiram documentação dedicada: o confinamento do pacote do motor bloqueava o acesso a dispositivos de entrada até a conexão explícita da permissão de joystick, e o servidor de áudio mantinha um estado de mudo por aplicativo que silenciava apenas o jogo. Ambos os casos foram diagnosticados e estão registrados como solução de problemas no material do projeto (Vianna, 2026a) — são exemplos de ameaças à reprodutibilidade que costumam passar em branco em relatos de jogos.
- **Empacotamento:** a geração de um executável independente (FR-033) estava bloqueada pela ausência dos modelos de exportação do motor no ambiente; o critério permanece aberto.

5.4 Implicações

Para quem projeta jogos educacionais, o caso sugere que a IA generativa pode entrar em cena como personagem consultivo — e não como agente — com latência de segundos e execução local, sem abrir mão de previsibilidade. Para quem pesquisa agentes em jogos, o desenho oferece um ponto de comparação: os mesmos modelos que rendem mais quando recebem autonomia também rendem valor narrativo quando recebem limites explícitos. Por fim, para a prática de desenvolvimento, o experimento sugere que um projeto de autor único assistido por agentes de IA pode manter rastreabilidade quando a especificação e os testes precedem a implementação — e quando cada promessa do resumo é verificável por um comando.

6 Conclusão

Este artigo descreveu o projeto, a implementação e a validação do Laboratório 404, um jogo 3D educacional/ficcional no qual uma IA local participa do mundo sem controlá-lo. O objetivo foi atingido no nível de protótipo: as seis provas de conceito foram concluídas e demonstráveis, a suíte automatizada cobre 210 a 211 verificações do motor e 16 testes do adapter, todas aprovadas, e o jogo roda a 60 FPS (limitados por *vsync*) em GPU integrada, mantendo conversa local com latência entre 12,9 s e 22,0 s.

As contribuições práticas são o padrão de *IA diegética com capacidade limitada* — processo separado, contrato estruturado e sanitização em duas camadas —, o processo incremental orientado a especificação com evidências automatizadas e visuais, e os artefatos públicos que permitem reproduzir e auditar cada afirmação quantitativa deste texto (Vianna, 2026a,b).

As limitações são conhecidas e declaradas: ausência de estudo com usuários, medições de latência em pequena amostra, validação manual de hardware pendente, empacotamento independente ainda não gerado e modelo-alvo menor não executado. Como trabalhos futuros, pretende-se: (i) executar e comparar o `llama3.2:3b` com o modelo usado aqui; (ii) concluir o empacotamento e a demonstração sem intervenção do desenvolvedor (FR-033); (iii) realizar o playthrough humano de ponta a ponta e, depois, um estudo controlado com usuários em contexto educacional; e (iv) ampliar o conteúdo do vertical slice mantendo a mesma disciplina de evidências.

Agradecimentos

O autor agradece a si mesmo — pela teimosia de levar este projeto até o fim.

Referências

- Clark C. Abt. *Serious Games*. The Viking Press, New York, 1970.
- Blender Foundation. *Blender 5.2 LTS Manual*, 2026. URL <https://docs.blender.org>. Acesso em: 10 set. 2026.
- Encode. Uvicorn Documentation, 2026. URL <https://www.uvicorn.org>. Acesso em: 10 set. 2026.
- FastAPI. FastAPI Documentation, 2026. URL <https://fastapi.tiangolo.com>. Acesso em: 10 set. 2026.
- Roberto Gallotta, Graham Todd, Marvin Zammit, Sam Earle, Antonios Liapis, Julian Togelius, and Georgios N. Yannakakis. Large Language Models and Games: A Survey and Roadmap. *IEEE Transactions on Games*, 2024. doi: 10.1109/TG.2024.3461510. Acesso antecipado; arXiv:2402.18659.
- James Paul Gee. *What Video Games Have to Teach Us About Learning and Literacy*. Palgrave Macmillan, New York, 2003.
- Godot Engine. *Godot Engine 4.5 Documentation*, 2026. URL <https://docs.godotengine.org>. Acesso em: 10 set. 2026.
- Khronos Group. glTF 2.0 Specification, 2026. URL <https://www.khronos.org/glTF/>. Acesso em: 10 set. 2026.
- Fedwa Laamarti, Mohamad Eid, and Abdulmotaleb El Saddik. An Overview of Serious Games. *International Journal of Computer Games Technology*, 2014:1–15, 2014. doi: 10.1155/2014/358152.

- Ollama. Ollama: Run Large Language Models Locally, 2026. URL <https://ollama.com>. Acesso em: 10 set. 2026.
- Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior, 2023. arXiv preprint.
- Pydantic. Pydantic Documentation, 2026. URL <https://docs.pydantic.dev>. Acesso em: 10 set. 2026.
- William da Silva Vianna. Laboratório 404 — repositório do projeto (código, especificação e evidências), 2026a. URL <https://github.com/wvianna/mcp-blender-vscode-godot-game>. Acesso em: 10 set. 2026.
- William da Silva Vianna. Vídeo de demonstração do Laboratório 404 (POC 2), 2026b. URL https://github.com/wvianna/mcp-blender-vscode-godot-game/blob/main/docs/videos/poc-laboratorio-404_1.mp4. 1 min 08 s; acesso em: 10 set. 2026.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models, 2023. arXiv preprint.
- Michael Zyda. From Visual Simulation to Virtual Reality to Games. *Computer*, 38(9):25–32, 2005. doi: 10.1109/MC.2005.297.